

Grow Coach (grow-za)

Project workflow, implemented features and how the system works

This document describes the code in `EmmanuelAzu/grow-za`, a fork of `kmo03/grow-za` taken at commit `de14da5` ("Add Grow coach selection and onboarding wizards", 2 Oct 2026). It is based on reading the source code. Nothing was run or deployed, so every behaviour below comes from the code, not from a live test.

In one sentence: Grow Coach is the front door to an AI leadership-coaching programme. A short *selection wizard* (`/apply`) filters applicants for real commitment, an *onboarding wizard* (`/`) creates their account and memory in the Pickaxe-hosted AI coach ("Coach Max"), and a set of email, calendar and cron services keep people engaged before and after they start.

Contents

1. What the product does

2. Architecture

3. End-to-end journey

4. Selection wizard (`/apply`)

5. Onboarding wizard (`/`)

6. Pickaxe integration

7. Email and calendar services
8. Follow-up reminders and cron

9. Pickaxe actions for Coach Max

10. API reference

11. Configuration

12. Access model and security

13. Tech stack and code layout

14. Gaps, risks and README drift

1. What the product does

The programme is a three-month leadership coaching journey delivered by an AI coach, with sessions roughly every fortnight (about 2 hours of commitment per fortnight, per the wizard's own wording). The company's stated problem is people who join without commitment and drop out, so this app puts a qualification step in front of the coach and a smooth hand-off behind it.

Capability	What it does	Where
Selection wizard	Four yes/no-style readiness questions with warm exits, a soft "I think so" nudge, acceptance, and the choice to start now or schedule onboarding.	<code>app/apply</code> , <code>components/apply</code> , <code>lib/apply-flow.ts</code>
Application capture	Records every outcome (passed or exited, and where) so drop-off can be analysed.	<code>app/api/applications</code>
Scheduling and invites	Applicant picks a slot; the server emails an HTML invite with an <code>.ics</code> calendar attachment through Resend. A browser-side <code>.ics</code> download is the fallback.	<code>app/api/invite</code> , <code>lib/send-invite.ts</code> , <code>lib/calendar.ts</code>

Follow-up reminders	Applicants who exit can ask to be nudged in 1 month to 1 year. A daily cron sends the reminder.	<code>app/api/follow-up</code> , <code>app/api/cron</code>
Onboarding wizard	Collects name, email and optional accountability buddy, checks if the person already onboarded, creates the Pickaxe account and writes memories, then redirects to the coach.	<code>/</code> , <code>app/api/onboarding</code>
Pickaxe integration	Server-side client for the Pickaxe Studio API: users, memory read and write.	<code>lib/pickaxe.ts</code>
Coach actions	Two endpoints that Coach Max calls from inside Pickaxe: book the next session, and email goals to the manager.	<code>*.pickaxe-action.yaml</code>
Branded email shell	One shared HTML email layout used by every message.	<code>lib/email-layout.ts</code>

2. Architecture

A single Next.js 16 application, deployed on Vercel, is the only code in this repository. It serves two browser wizards and a set of API routes. Everything else (the AI coach, user accounts, memory) lives in Pickaxe; email is sent through Resend. There is **no database** in the repo.

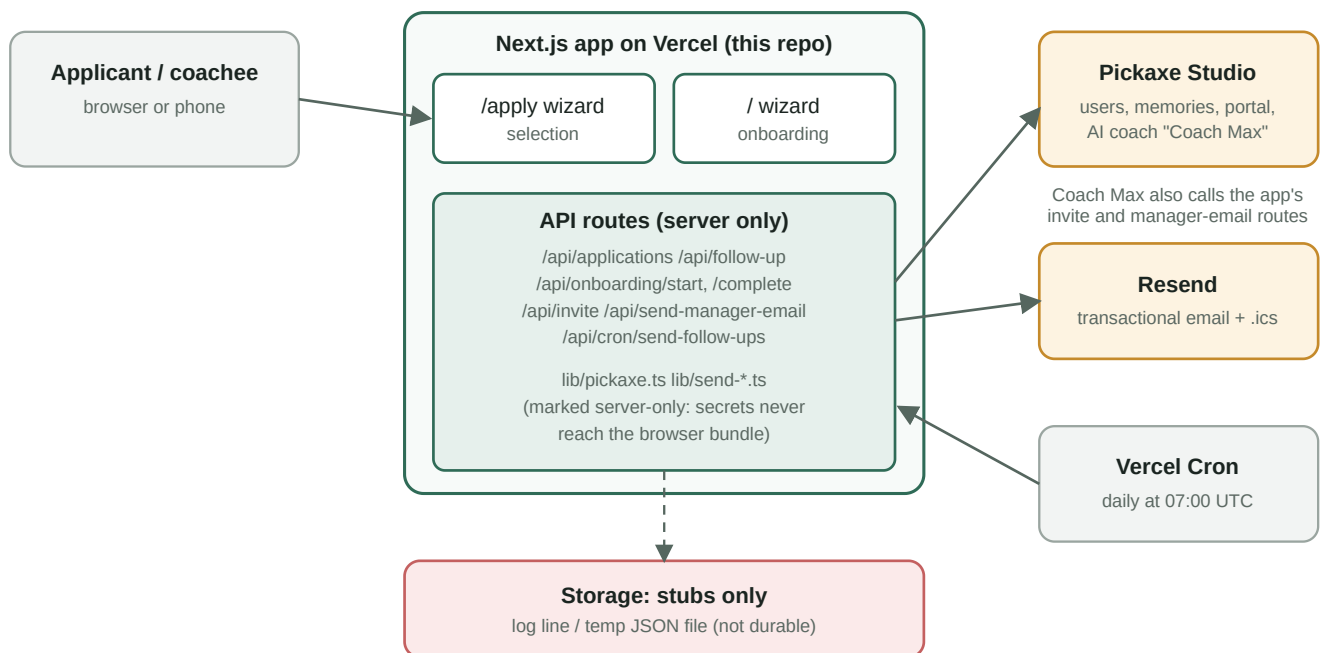


Figure 1. System architecture. Solid arrows are requests made or received by the app. After onboarding the browser is redirected straight into the Pickaxe portal, and Coach Max calls two app endpoints from within a conversation.

3. End-to-end journey

A person typically moves through four stages. The two wizards are separate pages, and the first can hand off to the second.

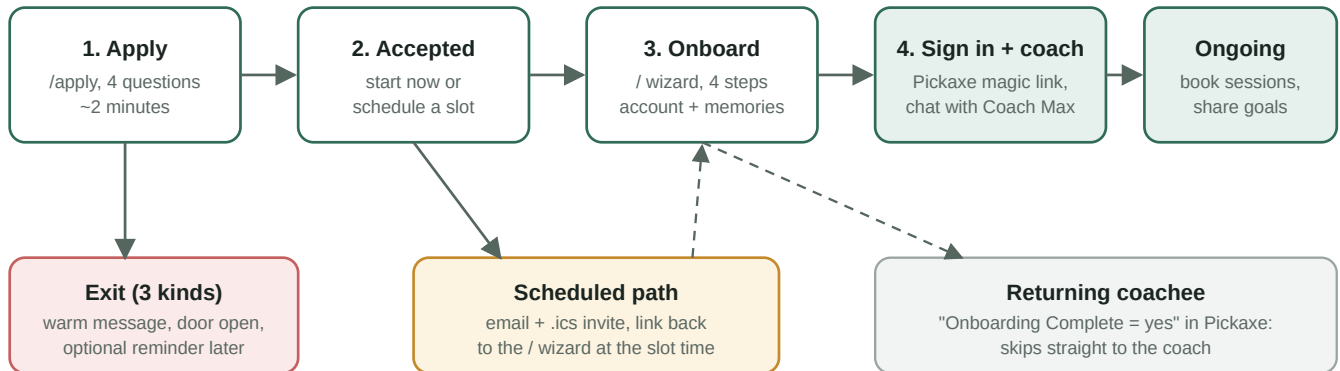


Figure 2. The coachee journey, with the three side paths (exit, scheduled, returning).

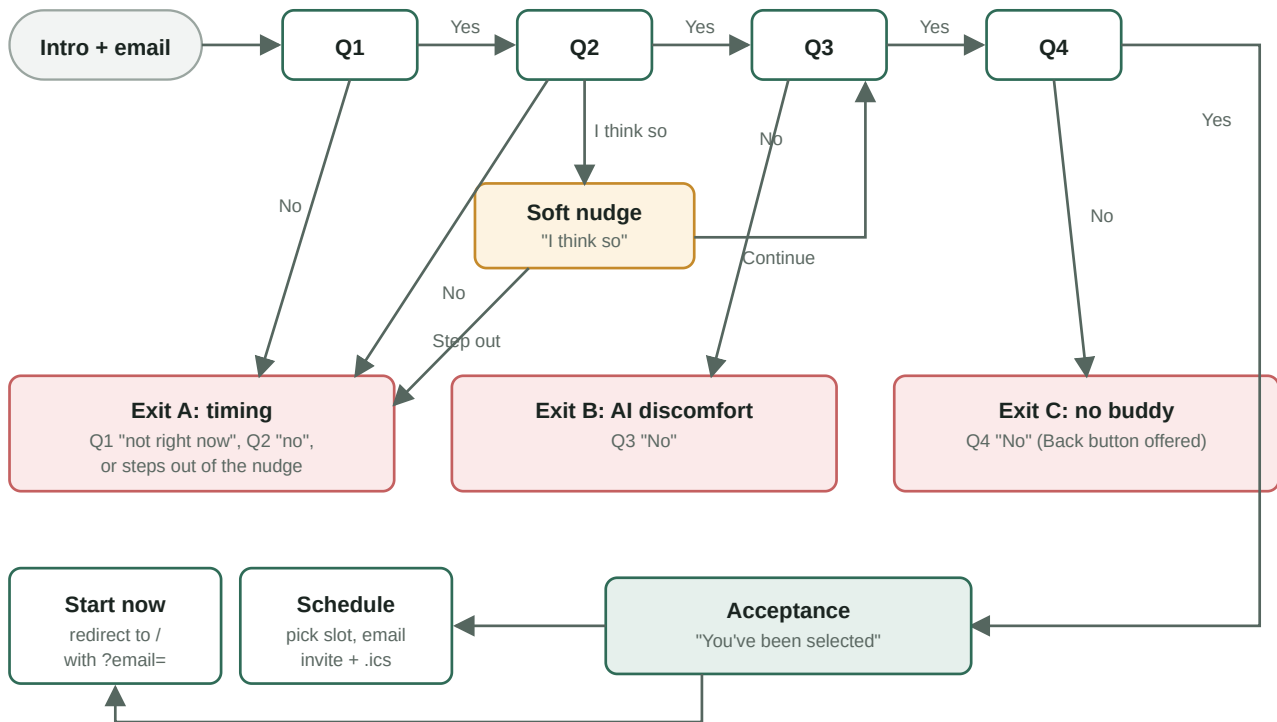
4. Selection wizard (/apply)

The wizard is a single client-side page driven by React state. Branches are not in the URL, so nobody can jump straight to the acceptance screen. Wording is held in `lib/apply-flow.ts` so it can be reviewed in one place (the code comments say it is the programme owner's text, verbatim).

4.1 The four questions

#	Heading	Question (abridged)	Answers
1	Leadership Growth Desire	Do you want to develop your career by growing yourself as a leader and becoming a more effective manager?	Yes / Not right now
2	Availability to Invest Time	Willing to invest 2 hours every fortnight for the next 3 months?	Yes / I think so / Realistically, no
3	Comfort with AI Coaching	Comfortable chatting with an AI-powered leadership coach?	Yes / No
4	Accountability Buddy	Willing to share your goal with a buddy (manager, friend, peer) who helps hold you accountable?	Yes / No

4.2 Decision tree



Every outcome (pass or exit) is recorded via POST /api/applications; exits also offer a follow-up reminder form.

Figure 3. Selection decision tree. Q1, Q3 and Q4 are binary; Q2 has a third "maybe" branch that gets a deliberate off-ramp.

4.3 Behaviours worth knowing

- **Exits are framed as successes.** There are three exit messages (timing, AI discomfort, no buddy). Each leaves the door open and tells the person to ask their manager to reapply later.
- **Back navigation.** A history stack lets people step back (Q3 can be reached directly or via the nudge). From an exit screen, Back returns to the question that caused it, with the previous answer kept. The exit that already happened stays recorded, so someone who reconsiders produces two rows, by design.
- **Progress.** Only the four questions show a progress indicator (the nudge keeps Q2's position); intro, exits and scheduling sit outside it.
- **Email on the opening screen is optional** and is not a gate. It is carried through to the acceptance, scheduling and onboarding hand-off screens.
- **Capture is fire-and-forget.** `recordApplication()` never throws or blocks the UI; a failed write costs one analytics row, not the applicant's experience.
- **Start now** navigates to the onboarding URL (`NEXT_PUBLIC_ONBOARDING_URL` , default `/`) with `?email=` appended so the first field arrives filled.
- **Schedule for later** opens a date/time picker (defaults to tomorrow morning, advises 15 to 30 uninterrupted minutes). The booking is recorded (`PATCH /api/applications`), then `POST /api/invite` emails the invite. If email fails, the booking is kept and the person can download the `.ics` file built in the browser instead.

4.4 Exit follow-up offer

Every exit screen offers: "Would you like me to reach out to you in a few weeks or months' time?" The applicant gives a first name, email and a timeframe (1, 2, 3, 6 months or 1 year). See section 8 for how this

is delivered.

5. Onboarding wizard (/)

The onboarding wizard is a state-driven page with four steps. It reads `?email=` on the server (`app/page.tsx`) so a handed-over address is in the initial state on first paint.

#	Step id	What the user sees / what happens
1	<code>email</code>	Name (required), email (required), and optional accountability buddy name and email. On submit, calls <code>POST /api/onboarding/start</code> . If the account is already marked complete, the browser redirects straight to the coach; otherwise it moves to step 2.
2	<code>welcome</code>	Explains the two levels of coaching: a 3-month growth goal, and in-session challenges linked back to that goal.
3	<code>emailAuth</code>	Explains the Pickaxe magic-link sign-in coming next. Shows the email back with a "Change it" link (the last chance to catch a typo). Its Next button triggers the save.
4	<code>completion</code>	Spinner ("Setting up your coach..."). Calls <code>POST /api/onboarding/complete</code> , then redirects to <code>NEXT_PUBLIC_PICKAXE_BOT_URL</code> . On error it offers "Try again" and keeps the answers.

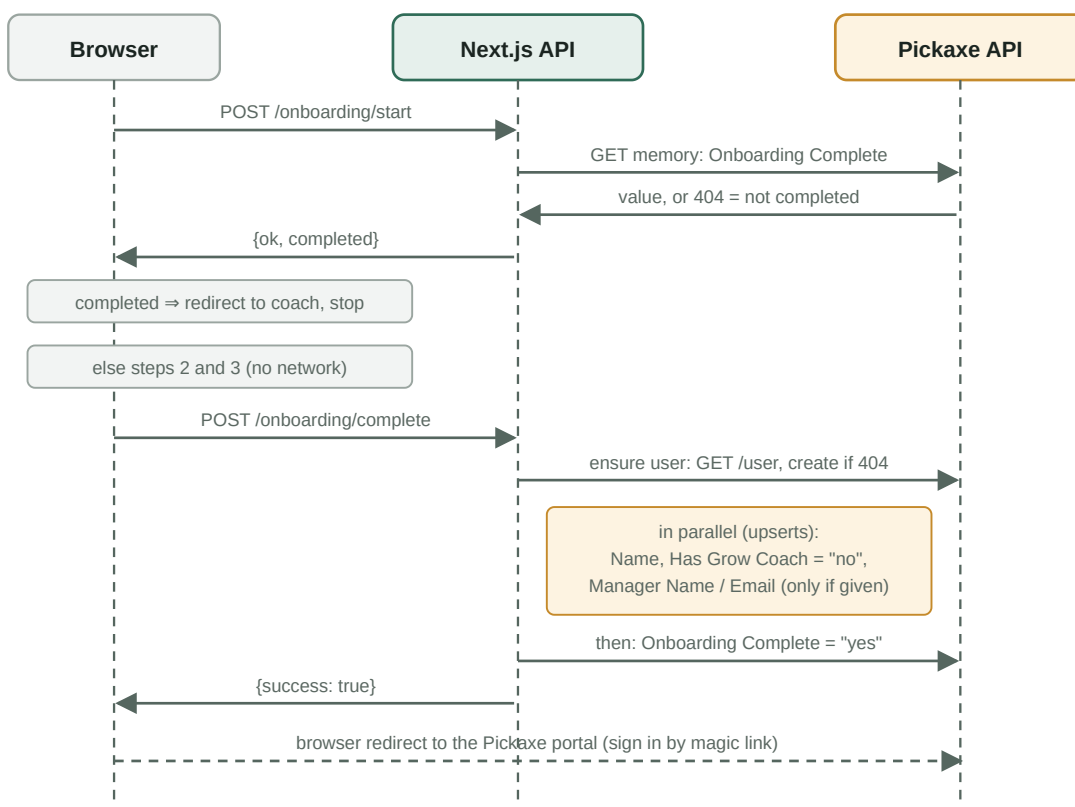


Figure 4. Onboarding request sequence. The completion flag is written last, so it never flips before the context behind it is stored.

6. Pickaxe integration

`lib/pickaxe.ts` is the only code that talks to Pickaxe. It is marked `server-only`, so importing it in a client component is a build error and `PICKAXE_API_TOKEN` cannot reach the browser. Config is read at call time, so

a missing variable becomes a handled request error rather than a build failure.

6.1 Memories the app reads and writes

Memory (env var)	Value	Written
Name (PICKAXE_NAME_MEMORY_ID)	Coachee's name	Every completion; overwrites
Manager Name / Manager Email	Accountability buddy (often the manager); names pre-date the "buddy" wording	Only when provided, so a re-run never blanks existing values
Has Grow Coach	"no" (constant: everyone is now self-directed)	Every completion
Onboarding Complete	"yes"	Last, after the others succeed
Coaching Goals (PICKAXE_COACHING_GOALS_MEMORY_ID)	Structured goals block	Currently not written (see section 14)

6.2 Helpers

- `getUserMemory(email, memoryId)` fetches one memory server-side with `?memoryId=` (the unfiltered list paginates at 10 rows and the workspace has more memory types than that). A 404 means "nothing stored", not an error.
- `ensureStudioUser(email, name?)` looks the user up and creates them with the configured access group if Pickaxe has never seen them. The access group is what lets them past the invite-only portal. A "user already exists" race is resolved by re-checking.
- `setUserMemory(email, memoryId, value)` posts to `/memory/user/create` (which upserts) and falls back to `PATCH` with the value wrapped in `{"data":{"value":...}}`.
- `formatGoalsMemory()` builds the goals block and appends it under a dated heading if goals already exist, idempotently. It is implemented and ready, but nothing calls it now.
- Error bodies from Pickaxe are truncated to 200 characters; the token is never logged or returned.

Design choice: because onboarding runs before first sign-in and Pickaxe cannot write memories for an unknown user, this app creates the Pickaxe user itself instead of sending an invite. A Pickaxe invitee does not exist as a user until they sign in, and Pickaxe will not email someone who already exists, so do not send Pickaxe invites to people who use this link.

7. Email and calendar services

All email is sent through Resend from `coach@grow-za.com` (domain verified in `eu-west-1`). Replies go to `REPLY_TO_ADDRESS` (during the pilot, a named human who handles replies by hand so reply types can later inform an automated handler). Each message shares one branded layout (`lib/email-layout.ts`) and an inline logo attached by content-id, and has both HTML and plain-text bodies.

Email	Trigger	Details
-------	---------	---------

Onboarding invite	Applicant schedules onboarding in <code>/apply</code> (<code>POST /api/invite</code>)	Subject "Your Grow AI Coach onboarding session". States the slot in the applicant's time zone (falls back to Africa/Johannesburg), has a "Start my onboarding" button, prep advice, and a <code>METHOD:REQUEST .ics</code> with the sender as organiser and the applicant as attendee (RSVP requested) so Gmail and Outlook show accept/decline inline. 30-minute event.
Coaching-session invite	Coach Max books the next session (<code>POST /api/invite</code> with <code>inviteType: "coaching-session"</code>)	"Your next session with Coach Max". 60-minute event with reminders 24 hours and 1 hour before. Links to the Pickaxe coaching portal.
Manager email	Coach Max, after the coachee approves a draft (<code>POST /api/send-manager-email</code>)	Sent as "Coach Max" to the manager, with the coachee on CC, subject "Coaching goals for {name}". Body is the approved text, split into paragraphs; footer notes it was shared with the coachee's approval.
Follow-up reminder	Daily cron, for due requests	"Revisiting your interest in leadership growth through coaching", with a button back to <code>/apply</code> .

Validation details: the invite endpoint rejects bad emails and unparseable or past times (one minute of clock-skew slack), accepts a time zone only if the runtime recognises it, and caps names at 120 characters. Resend failures are logged server-side and the caller gets a plain sentence with HTTP 502, so the wizard can fall back to the local download.

8. Follow-up reminders and cron



Cron marks a record sent only after a successful send; a failure stays due and retries the next day.

Figure 5. The follow-up pipeline.

- `POST /api/follow-up` requires a first name (max 80 chars), a valid email, and a known timeframe. It stores `{id, firstName, email, requestedAt, sendAfter, timeframeLabel, sent, sentAt}`.
- `GET /api/cron/send-follow-ups` (scheduled in `vercel.json` as `0 7 * * *`) refuses every request unless `CRON_SECRET` is set and matches, using a length check plus an XOR comparison. It sends each due, unsent record in turn; one bad address does not stop the batch. It returns `{due, sent, failed}`.
- If a send succeeds but marking it sent fails, the code logs loudly because the next run could send it twice.

9. Pickaxe actions for Coach Max

Two YAML files in the repo define "actions" that you import into Pickaxe. Each is a small Python function that POSTs to this app, so Coach Max can trigger real-world effects from inside a conversation.

Action	When Coach Max uses it	Calls
<code>send_calendar_invite</code>	Only after the coachee agrees a specific date and time. It must confirm the time in plain language first, send ISO-8601 with the South Africa offset (<code>+02:00</code>), and pass email and first name.	POST <code>{INVITE_ENDPOINT_URL}</code> with <code>inviteType: "coaching-session"</code>
<code>send_manager_email</code>	Only after the coachee explicitly approves a draft email sharing their coaching goals with their manager, written in Coach Max's voice.	POST <code>{MANAGER_EMAIL_ENDPOINT_URL}</code>

Both return a sentence for the coach to read ("Invite sent...", or an apology and fallback wording) instead of failing silently. The endpoints themselves cannot verify that the coachee approved anything; they send what they are given.

10. API reference

Route	Method	Purpose and notes
<code>/api/applications</code>	POST	Record an application outcome. Validates answers against allow-lists (unknown values are dropped, not stored); forces <code>exitStage = null</code> when passed. Returns <code>{ok, id}</code> . Storage is a log-line stub.
<code>/api/applications</code>	PATCH	Attach <code>scheduledFor</code> to an existing application id.
<code>/api/follow-up</code>	POST	Save a "remind me later" request (section 8).
<code>/api/invite</code>	POST	Send an onboarding or coaching-session invite email with <code>.ics</code> . 15 s max duration.
<code>/api/send-manager-email</code>	POST	Email a manager the coachee's approved goals. Body capped at 20,000 characters.
<code>/api/onboarding/start</code>	POST	Return <code>{ok, email, completed}</code> . A Pickaxe outage returns 502 rather than reporting "not completed", so nobody is re-onboarded because of an outage.
<code>/api/onboarding/complete</code>	POST	Create the Pickaxe user and write memories (section 5). Returns <code>{success}</code> or an error with 4xx/502.
<code>/api/cron/send-follow-ups</code>	GET	Secret-protected daily job (section 8).

11. Configuration

Variable	Scope	Purpose
----------	-------	---------

PICKAXE_API_TOKEN	Server	Bearer credential for the Pickaxe Studio API.
PICKAXE_NAME / MANAGER_NAME / MANAGER_EMAIL / ONBOARDING_COMPLETE / HAS_GROW_COACH / COACHING_GOALS _MEMORY_ID	Server	IDs of the six memory types (all six are required by <code>getMemoryIds()</code> , even though goals are not written now).
PICKAXE_ACCESS_GROUP_ID	Server	Access group given to accounts the app creates, which grants portal entry.
NEXT_PUBLIC_PICKAXE_BOT_URL	Public	Where users are redirected after onboarding: <code>https://studio.pickaxe.co/{studioId}/{portalPath}</code> .
NEXT_PUBLIC_ONBOARDING_URL	Public	Where accepted applicants go on "start now" (default <code>/</code>).
ONBOARDING_URL	Server	Absolute link written into invites. Falls back to the Vercel production URL, then the request origin.
RESEND_API_KEY , RESEND_FROM_ADDRESS , REPLY_TO_ADDRESS	Server	Email sending. The from-address must be on a Resend-verified domain; <code>REPLY_TO_ADDRESS</code> is optional.
CRON_SECRET	Server	Required for the cron endpoint to run at all.
WIZARD_URL	Server	Optional override for the follow-up email's link to <code>/apply</code> .
PICKAXE_API_BASE_URL , FOLLOW_UP_STORE_PATH	Server	Local-testing overrides (stand-in Pickaxe server; location of the temp JSON store).

Utility: `clear-test-user.sh <email>` blanks (or, with `DELETE_MODE=1`, deletes) all of a test user's Pickaxe memory values so the brand-new-user flow can be re-tested with the same address. It checks every HTTP status so a failure is never reported as success.

12. Access model and security

12.1 Deliberate decisions

- **Open access by URL.** There is no login, invite check or signed link; the email is taken on trust. The README says this was chosen because the pilot client is non-technical.
- **Obscurity limits spread.** `app/robots.ts` disallows all crawling, and the root layout sets `noindex`, `nofollow` page metadata.
- **Typo mitigation.** The email is shown back before anything is written, with a "Change it" link.
- **Server-only secrets.** `lib/pickaxe.ts` and the email modules import `server-only`; `.env*` files are git-ignored except `.env.example`.
- **Input hygiene.** Email format is validated on client and server; names and bodies are length-capped; allow-lists guard stored answers; provider error text is kept in server logs and not echoed to users.

12.2 Consequences to be aware of

- Anyone with the URL can create a Pickaxe account for any address (Pickaxe usage is metered) and can overwrite a real coachee's Name and Manager memories, which the AI coach treats as context.
- `/api/invite` and `/api/send-manager-email` are public and unauthenticated. Anyone who finds them can make the platform send branded email from the verified domain to arbitrary addresses (the manager endpoint also takes free-form body text). The Pickaxe actions call them with no shared secret. Adding a secret header checked by both would close this.
- `/api/onboarding/start` reveals whether an email address has completed onboarding.

13. Tech stack and code layout

Stack: Next.js 16.3 (App Router), React 19.2, TypeScript 5, Tailwind CSS 4 with shadcn/ui and Radix, framer-motion for step transitions, lucide-react icons, and the Resend SDK. Deployed on Vercel (with Vercel Cron).

```
app/  
  page.tsx                onboarding page (reads ?email=)  
  apply/page.tsx          selection wizard (client state machine)  
  api/applications/       capture + schedule update  
  api/follow-up/          reminder request  
  api/invite/             onboarding / coaching-session invites  
  api/send-manager-email/  manager goals email  
  api/onboarding/start|complete/  
  api/cron/send-follow-ups/ daily reminder job  
  robots.ts              disallow all crawlers  
components/  
  apply/                  intro, question, nudge, exit, acceptance, schedule, scheduled,  
follow-up form  
  steps/                  onboarding screens (email, welcome, emailAuth, completion + 4  
unused, see 14)  
  onboarding-wizard.tsx   onboarding state machine  
  ui/                     shadcn components  
lib/  
  apply-flow.ts, apply-types.ts  selection copy, branching, types  
  pickaxe.ts               Pickaxe client (server-only)  
  send-invite.ts, calendar.ts    invite emails + ICS builder  
  send-manager-email.ts, send-follow-up.ts, email-layout.ts, email-config.ts, email-logo.ts  
  application-store.ts, follow-up-store.ts  storage stubs  
  app-urls.ts, onboarding-link.ts, datetime.ts, email.ts, steps.ts, types.ts, utils.ts  
*.pickaxe-action.yaml      two actions to import into Pickaxe  
clear-test-user.sh         test-user reset script  
vercel.json                cron schedule
```

The app is small (about 6,200 lines of TypeScript and CSS excluding the lockfile and images) and the code is heavily commented with the reasons behind decisions.

14. Gaps, risks and README drift

Several of these were found by comparing the README with the code. The code is the source of truth here, and the author's own comments flag the first three.

#	Finding	Detail and impact
1	Coaching goals are never saved	<code>onboarding/complete</code> carries a warning: the four goal-setting questions were removed on the assumption that a separate "Goal Setting" agent would run after account creation, but no such agent exists in the studio. Until one is built, the Coaching Goals memory stays empty and Coach Max opens its first session without goals. <code>formatGoalsMemory</code> and <code>GoalsData</code> remain in the code unused.
2	No durable storage	<code>application-store.ts</code> only logs, so conversion and drop-off data is not retained. <code>follow-up-store.ts</code> writes to <code>/tmp</code> , which on Vercel is per-instance and discarded, so follow-up reminders will not actually be sent in production until a real database is wired in. Both files are clean single seams to replace (three functions for follow-ups, two for applications).
3	README is out of date	It describes a 7/8-step onboarding with goal-setting questions, a coach check and a session-expectations screen, a <code>GET /api/onboarding/status/[email]</code> route, and "six" env vars. The code has a 4-step onboarding (email, welcome, emailAuth, completion), a <code>POST /api/onboarding/start</code> route, buddy name/email fields and more variables. It also omits the whole <code>/apply</code> flow, invites, follow-ups and Pickaxe actions.
4	Dead components	<code>coach-check-step</code> , <code>coach-confirmation-step</code> , <code>goals-step</code> and <code>expectations-step</code> are no longer imported by the wizard.
5	Stale reference	A comment in <code>ensureStudioUser</code> cites signed links in <code>lib/invite-token.ts</code> , which does not exist, so the claimed "only a Grow-issued token reaches this code" protection is not in place.
6	Unauthenticated email endpoints	See 12.2. This is the most important security follow-up.
7	Hardcoded portal URL	<code>send-invite.ts</code> embeds a specific Pickaxe studio and portal id for coaching-session invites rather than reading an environment variable.
8	Env config mismatches	<code>.env.example</code> says "six memories" but lists seven memory-related variables plus the access group; <code>getMemoryIds()</code> requires all six memory IDs, so a deployment must set Coaching Goals even though nothing writes it. The Resend key line in <code>.env.example</code> is blank, which is correct, but the file is committed, so keep real values out of it.
9	No automated tests	No test files or test script exist; <code>package.json</code> has only dev, build, start and lint.
10	Framework note	<code>AGENTS.md</code> warns this Next.js version has breaking changes and tells AI agents to read <code>node_modules/next/dist/docs/</code> before writing code. Run <code>npm install</code> first if you plan to change code.

Suggested next steps, in priority order:

1. Add a durable database (applications and follow-ups), or follow-up reminders will silently never send.
2. Build or wire the Goal Setting step so the Coaching Goals memory is populated, or restore the goals questions in onboarding.
3. Protect `/api/invite` and `/api/send-manager-email` with a shared secret that the two Pickaxe actions send.
4. Refresh the README to match the current flow, and delete the unused step components.

Prepared from a read-through of commit `de14da5`. Not executed or deployed; no live API calls were made.